Manage terraform Pipeline from Jenkins

Table Of Contents

RESOURCE INFORMATION **3**

DESCRIPTION 3
PREREQUISITES 3

INSTALLATION OF JENKINS. **4**

DESCRIPTION 4
PROCEDURE 4

CREATION OF JENKINS' SECRET. **7**

DESCRIPTION 7
PROCEDURE 7

CONFIGURATION OF SHARED LIBRARY **9**

DESCRIPTION 9
PROCEDURE 9

GITHUB CONFIGURATION **11**

TERRAFORM PIPELINE REPOSITORY 11
DESCRIPTION 11
PROCEDURE 11

JENKINS JOB CONFIGURATION **14**

DESCRIPTION 14
PROCEDURE 14

RESOURCE INFORMATION

DESCRIPTION

This document guides you on how to deploy the Terraform templates using Jenkins Pipeline.

PREREQUISITES

- An AWS Account with the user having the required permissions to create an EC2 instance.
- Create an EC2 instance with Ubuntu 20.04 version or above.
- On the EC2 instance the user must have installed Jenkins, Git, AWS CLI v2, unzip.
- The EC2 instance should have the IAM roles for required AWS services that would be deployed using the Terraform template.
- The user should have generated GitHub tokens for Application and Jenkins Pipeline GitHub Repo.

INSTALLATION OF JENKINS.

DESCRIPTION

This section will show how to install Jenkins and Terraform.

PROCEDURE

Step1: SSH into the Ubuntu EC2 instance and switch over to root user.

```
ubuntu@ip-172-31-159-3:~$ sudo -i
root@ip-172-31-159-3:~# cd /home/
root@ip-172-31-159-3:/home# ll
total 12
drwxr-xr-x  3 root   root   4096 Mar 26 07:16 ./
drwxr-xr-x 19 root   root   4096 Mar 26 07:16 ../
drwxr-xr-x  4 ubuntu ubuntu 4096 Mar 26 07:17 ubuntu/
root@ip-172-31-159-3:/home#
```

Step2: run **"apt-get update -y"** to update ubuntu.

```
root@ip-172-31-159-3:/home# apt-get update
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu focal-security InRelease
Reading package lists... Done
```

Step3: run the command **"java -version"** to check the installed Java version. If you receive below error then proceed with the Step 3 (a) or else proceed with Step4.

```
root@ip-172-31-159-3:/home# java -v

Command 'java' not found, but can be installed with:

apt install default-jre              # version 2:1.11-72, or
apt install openjdk-11-jre-headless  # version 11.0.20.1+1-0ubuntu1~20.04
apt install openjdk-13-jre-headless  # version 13.0.7+5-0ubuntu1~20.04
apt install openjdk-16-jre-headless  # version 16.0.1+9-1~20.04
apt install openjdk-17-jre-headless  # version 17.0.8.1+1~us1-0ubuntu1~20.04
apt install openjdk-8-jre-headless   # version 8u382-ga-1~20.04.1
```

Step3 (a): Run the command **"apt install openjdk-11-jre-headless"** to install.

```
root@ip-172-31-159-3:/home# apt install openjdk-11-jre-headless
```

Step3 (b): Once the above command has been executed run the command **"java -version"** to verify the Java version.

```
root@ip-172-31-159-3:/home# java -version
openjdk version "11.0.22" 2024-01-16
OpenJDK Runtime Environment (build 11.0.22+7-post-Ubuntu-0ubuntu220.04.1)
OpenJDK 64-Bit Server VM (build 11.0.22+7-post-Ubuntu-0ubuntu220.04.1, mixed mode, sharing)
```

Step4: Once the Java has been installed open the link https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/jenkins-ubuntu-20-install-git-jdk-java-ci-cd which has a guide to install Jenkins on Ubuntu 20.04. Follow the steps to install Jenkins.

Step5: Once Jenkins has been installed run the command **"systemctl status Jenkins"** to check if Jenkins is running or not.

```
root@ip-172-31-159-3:/home# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2024-03-26 07:35:48 UTC; 17s ago
   Main PID: 14109 (java)
      Tasks: 44 (limit: 1126)
     Memory: 325.6M
     CGroup: /system.slice/jenkins.service
             └─14109 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Mar 26 07:35:11 ip-172-31-159-3 jenkins[14109]: 0dbd96603c3840f6ada90627eab83668
Mar 26 07:35:11 ip-172-31-159-3 jenkins[14109]: This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Mar 26 07:35:11 ip-172-31-159-3 jenkins[14109]: ************************************************************
Mar 26 07:35:11 ip-172-31-159-3 jenkins[14109]: ************************************************************
Mar 26 07:35:11 ip-172-31-159-3 jenkins[14109]: ************************************************************
Mar 26 07:35:48 ip-172-31-159-3 jenkins[14109]: 2024-03-26 07:35:48.899+0000 [id=30]        INFO        jenkins.InitReactorRunner$1#onAttained: Completed initializat>
Mar 26 07:35:48 ip-172-31-159-3 jenkins[14109]: 2024-03-26 07:35:48.926+0000 [id=22]        INFO        hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and r>
Mar 26 07:35:48 ip-172-31-159-3 systemd[1]: Started Jenkins Continuous Integration Server.
Mar 26 07:35:49 ip-172-31-159-3 jenkins[14109]: 2024-03-26 07:35:49.942+0000 [id=45]        INFO        h.m.DownloadService$Downloadable#load: Obtained the updated d>
Mar 26 07:35:49 ip-172-31-159-3 jenkins[14109]: 2024-03-26 07:35:49.944+0000 [id=45]        INFO        hudson.util.Retrier#start: Performed the action check updates>
```

Step6: Once Jenkins has been installed. Run the command **"vi /etc/sudoers"** and add the following line **"jenkins ALL=(ALL) NOPASSWD: ALL"**. Once it is added save the file.

```
# See the man page for details on how to write a sudoers file.
#
Defaults        env_reset
Defaults        mail_badpass
Defaults        secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL
jenkins ALL=(ALL) NOPASSWD: ALL

# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "#include" directives:

:wq!
```

Step7: Now run the command **"sudo su jenkins"** to switch over to Jenkins user. After switching over to Jenkins user run the command **"aws configure"**.

Note: Do not give Access Key Id and Secret Key. Only give the region of your choice and the Default Output as "json".

```
root@ip-172-31-159-3:/etc# sudo su jenkins
jenkins@ip-172-31-159-3:/etc$ aws configure
AWS Access Key ID [None]:
AWS Secret Access Key [None]:
Default region name [None]: ap-south-1
Default output format [None]: json
jenkins@ip-172-31-159-3:/etc$
```

Step8: After configuring the AWS Cli. Restart Jenkins by running the command **"systemctl restart jenkins"**.

```
root@ip-172-31-159-3:/etc# systemctl restart jenkins
root@ip-172-31-159-3:/etc# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2024-03-26 07:47:32 UTC; 19s ago
   Main PID: 14674 (java)
      Tasks: 44 (limit: 1126)
     Memory: 269.0M
     CGroup: /system.slice/jenkins.service
             └─14674 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080
```

Step9: To Install Terraform go to /home directory and follow the steps mentioned in the below link.
https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli

Step10: Once the above commands have been executed. Reboot the EC2 Instance.

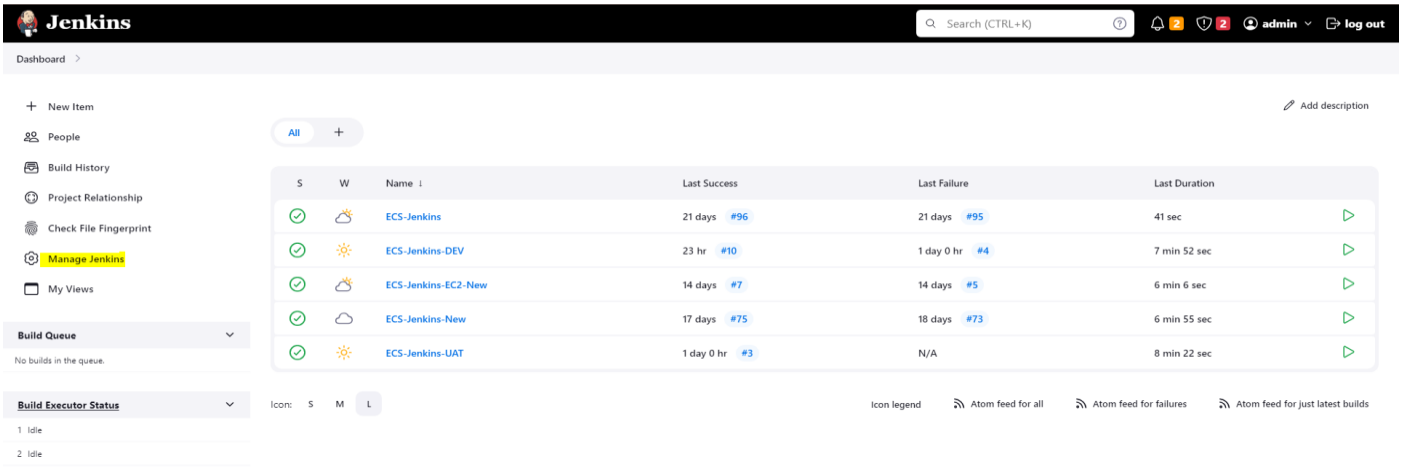
CREATION OF JENKINS' SECRET.

DESCRIPTION

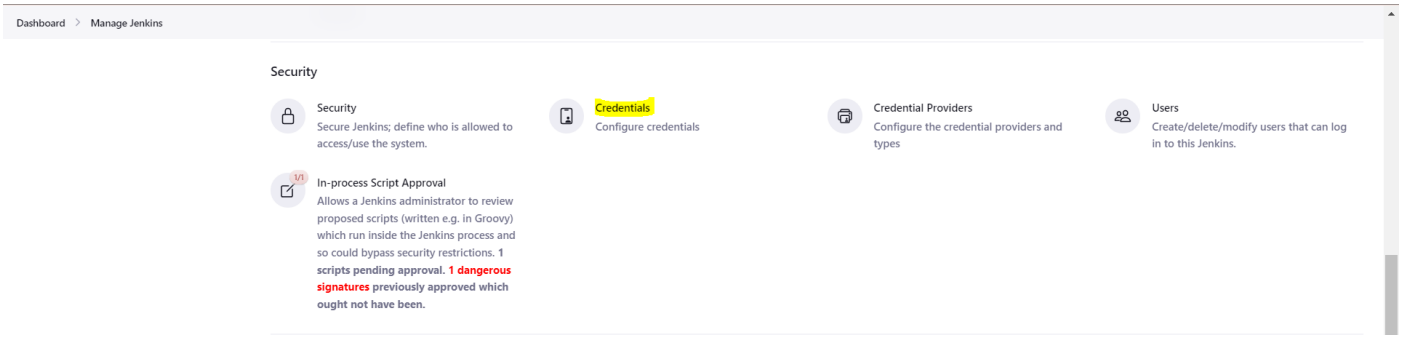
This section will show how to create Jenkins' Secrets.

PROCEDURE

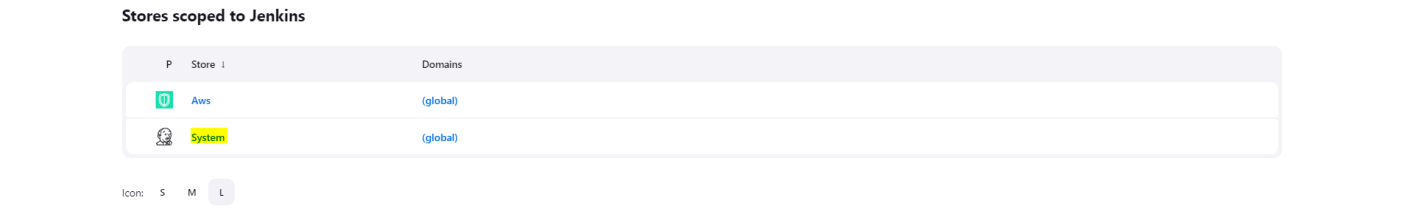
Step1: Go to Managed Jenkins in the Jenkins console.



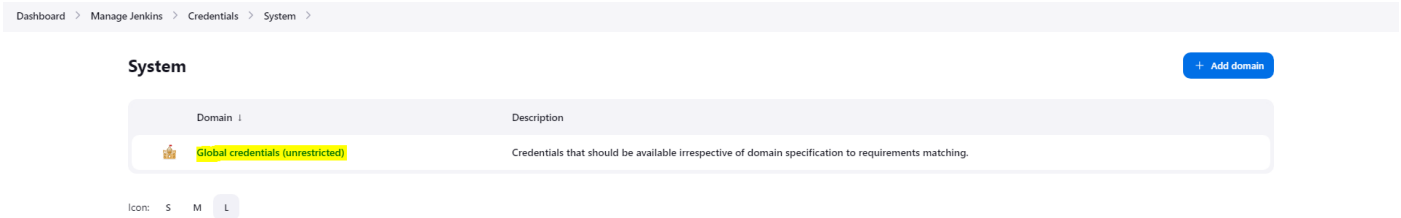
Step2: Go to Credentials configuration.



Step3: Click on Systems under the Stored Scoped to Jenkins.



Step4: Click on Global credentials.



Step5: Click on "Add Credentials"



Step6: For storing Git token of the Pipeline repositiory select kind as “Username with Password”. Give the Username anything of your choice and for the Password give the Git token that you have created and save it.



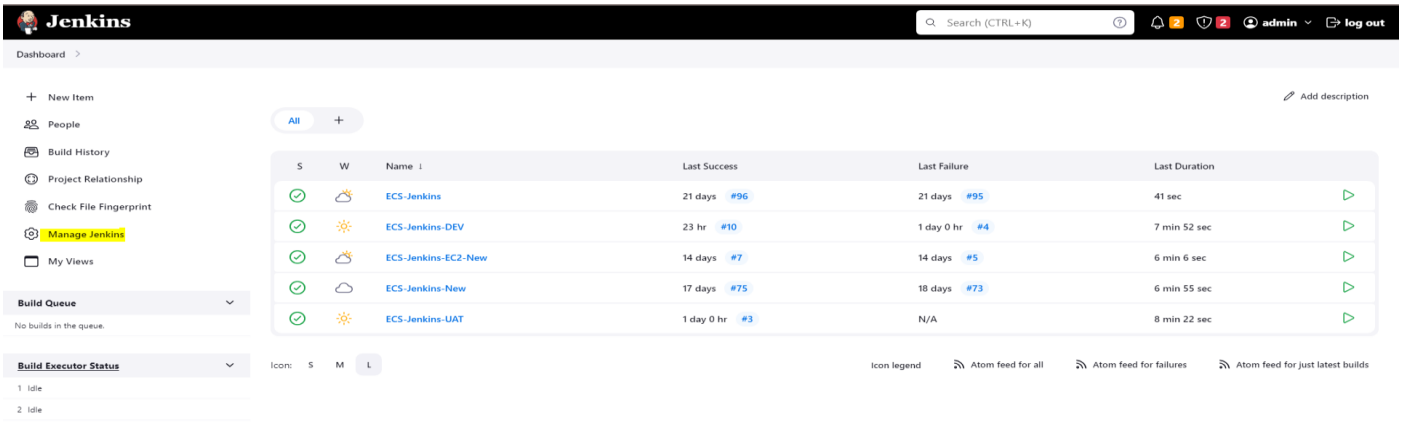
CONFIGURATION OF SHARED LIBRARY

DESCRIPTION

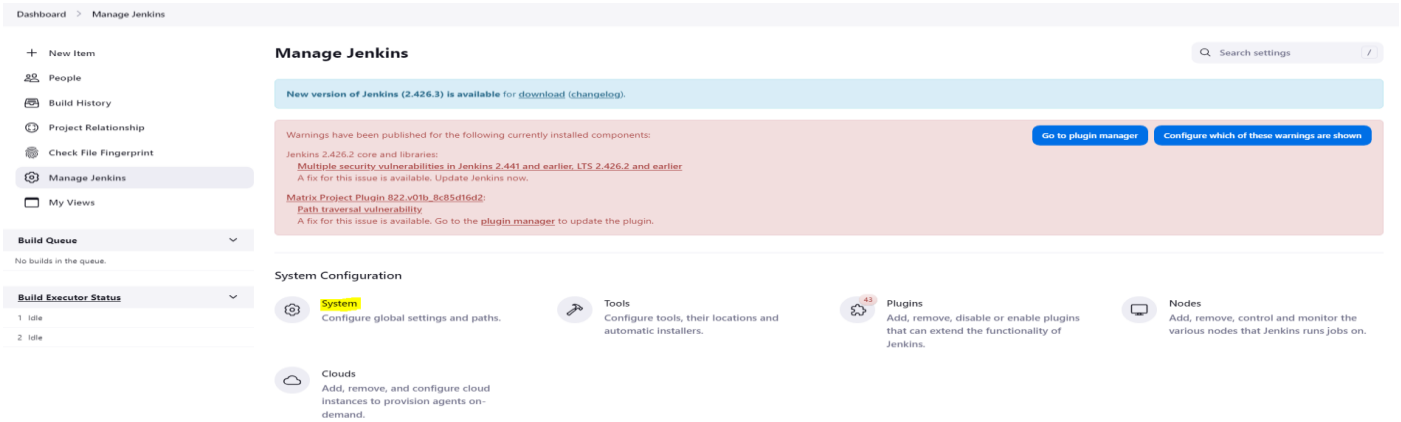
This section will show you how to create a Shared Library in Jenkins.

PROCEDURE

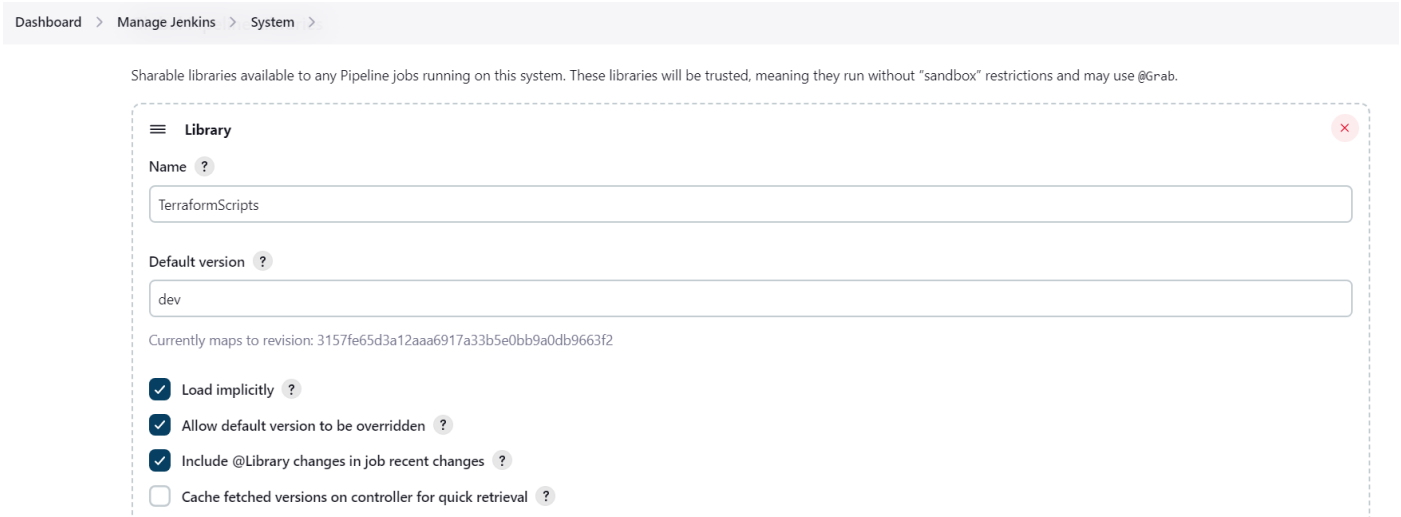
Step1: Go to Managed Jenkins in the Jenkins console.



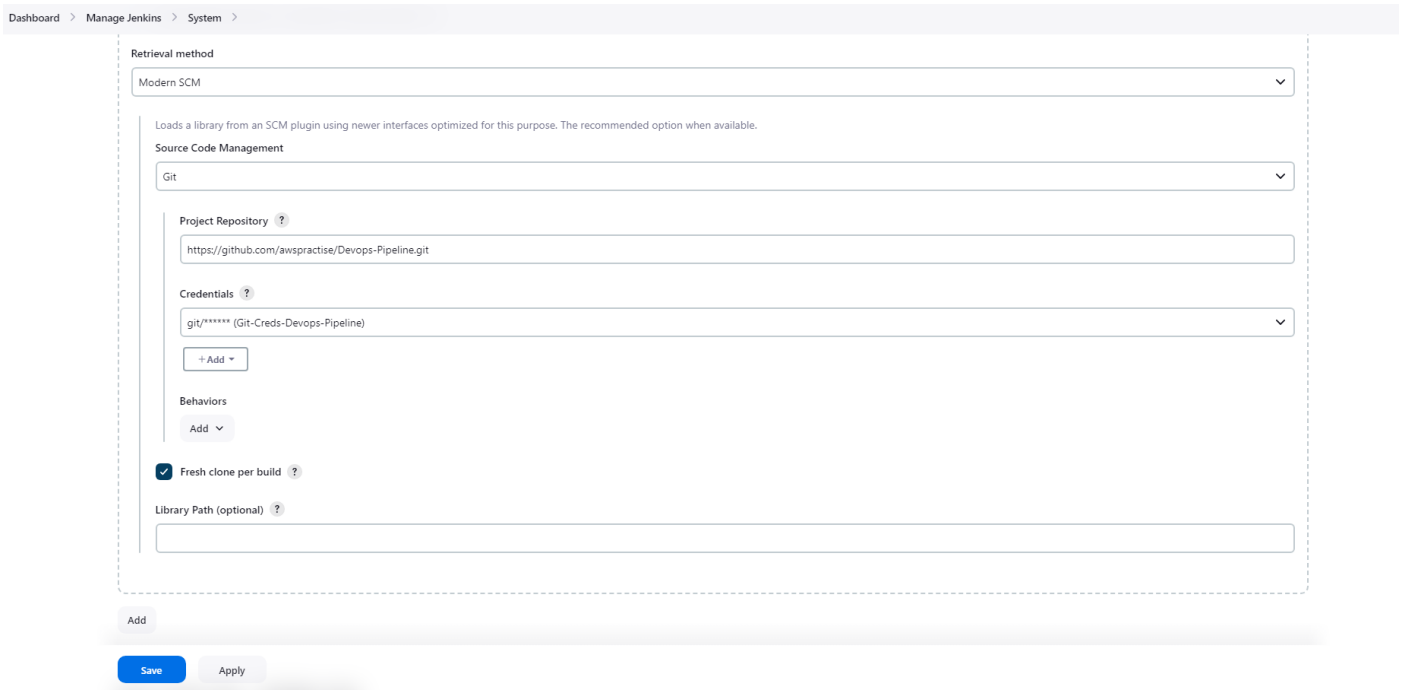
Step2: Go to System



Step3: Under Global Pipeline Libraries click on Add and give the Name as “TerraformScripts”. And in the Default version give the name of the git branch of the pipeline script and select “Load implcitily”, “Allow default version to be overridden” and “Include @Library changes in job recent changes.”



Step4: For Retrival Method select “Modern SCM” and in Source Code Management select Git. In the Project Repository give the Jenkins Terraform Pipeline Repo URL (Refer to Step1 from the GitHub Configuration section) and for the credentials select the appropriate credentials that you have created (Refer to Step 7 from Creation of Jenkins secret section for storing git credentials of the Terraform Pipeline repository).



Step5: Once everything is done click on “Save” to save the configuration.

GITHUB CONFIGURATION

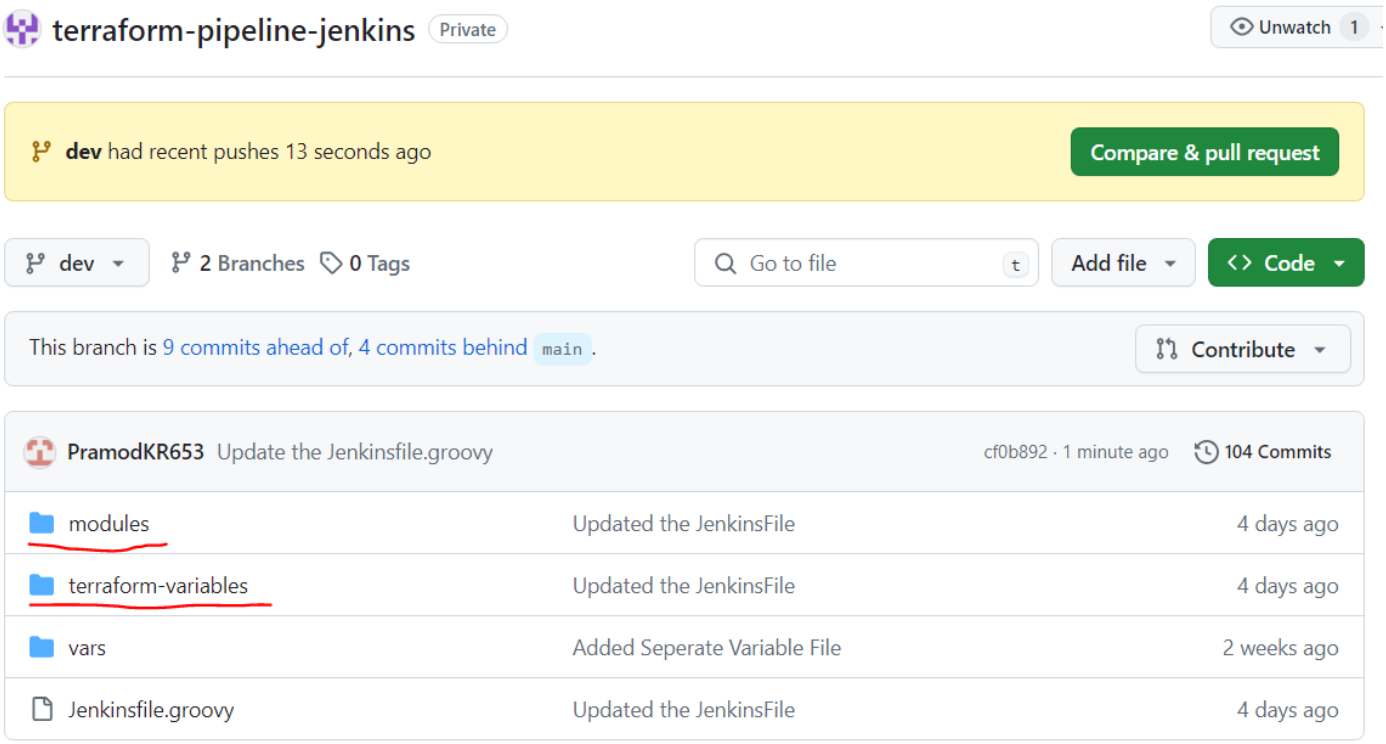
TERRAFORM PIPELINE REPOSITORY

DESCRIPTION

The following steps will show what needs to be uploaded to the Terraform pipeline GitHub repository.

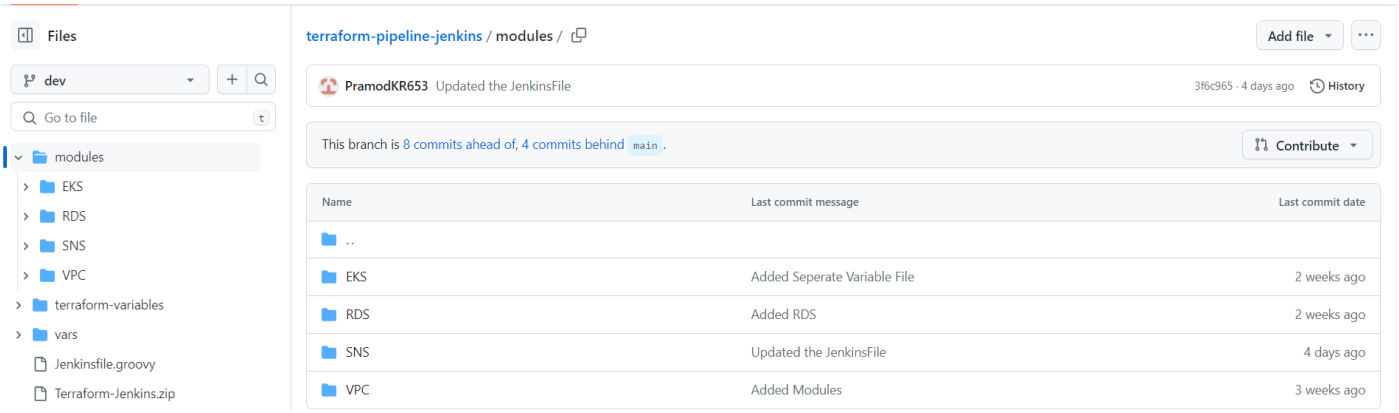
PROCEDURE

Step1: Create Two separate folders namely “modules” and “terraform-variables” or you can name anything for the two folders.

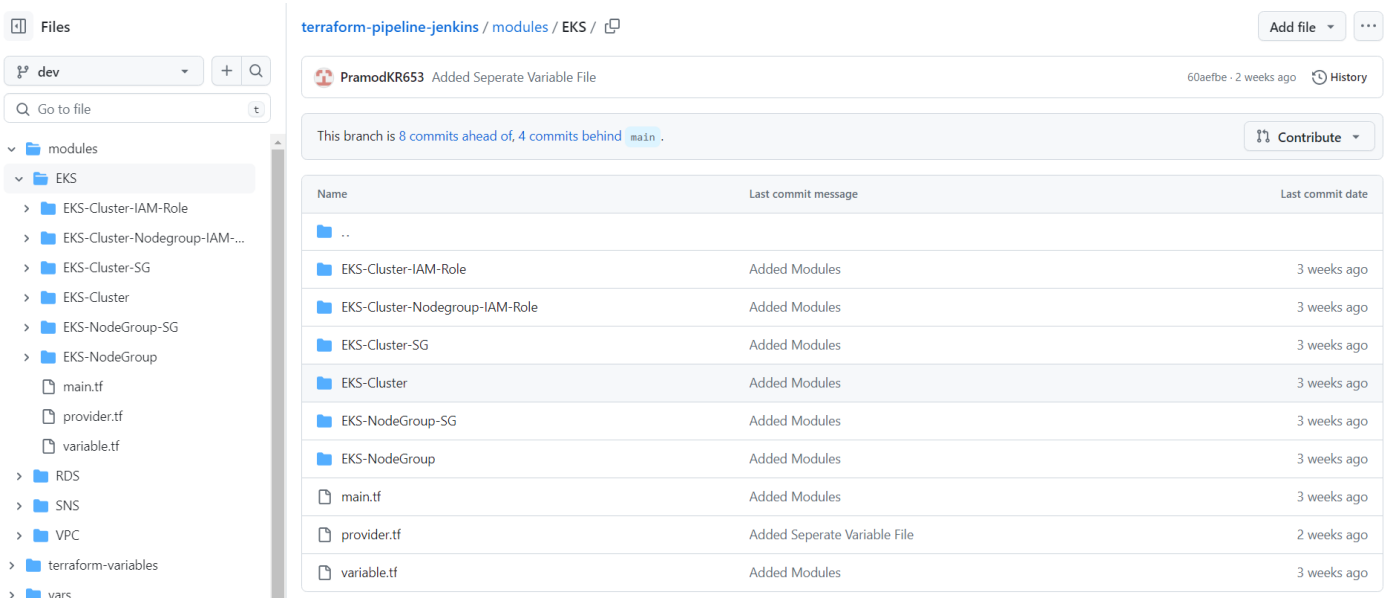


Step2: In the “modules” folder upload the Terraform templates to the folders based on the type of resource that you are deploying.

Ex: EKS, RDS, VPC etc.

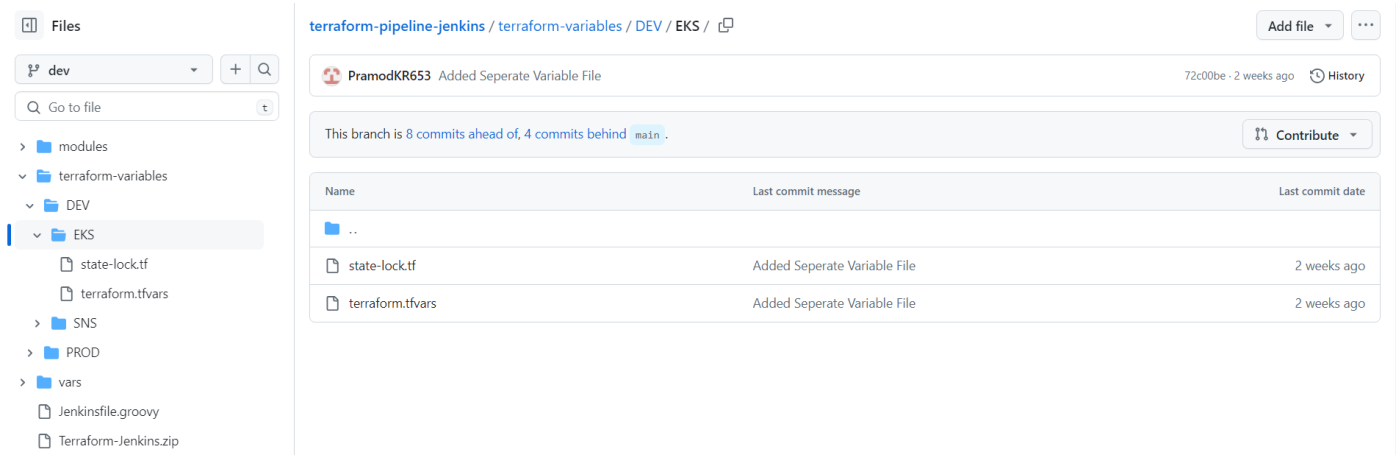


Step3: After creating the folder upload the terraform templates to the respective resource folder but **make sure not to upload the state-lock.tf and terraform.tfvars** files in the resource folder.



Note: The above screenshot is an example of Terraform Modules. If you have not created the Terraform templates as modules it is also fine, you can upload the Non moduli zed templates and execute them in the Terraform Pipeline.

Step4: Now Create subfolders in the "terraform-variables" folder according to your environments namely DEV, PROD etc. And in those subfolders create the Folders based on the Resource name like EKS, RDS etc and upload the terraform.tfvars and state-lock.tf files into it. Look into the below screenshot to get an idea.

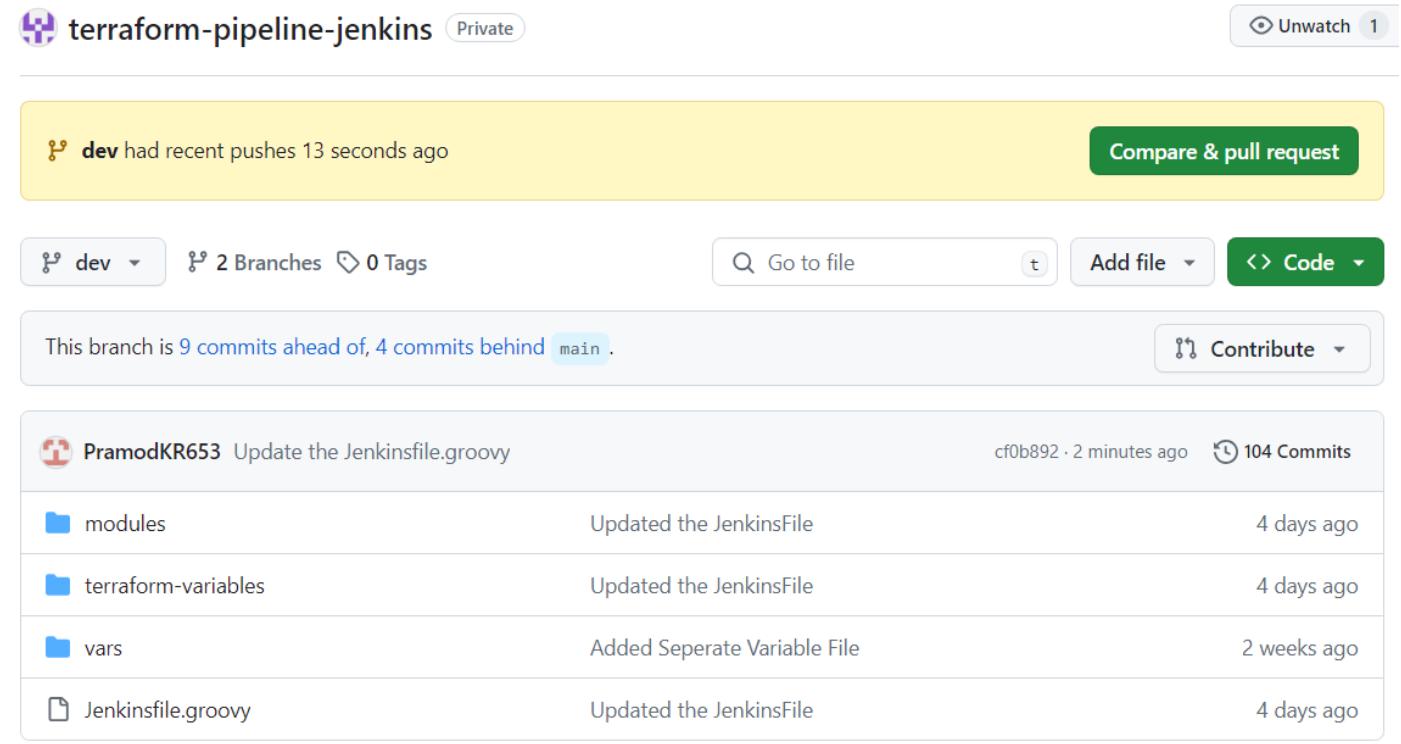


Step5: Now open the Jenkinsfile.groovy which is in the zip file. If you want to deploy the resources which are in the different AWS account, then you need to create a cross-account role in the other AWS account in which you want to deploy the resource using terraform and then you need to add the AWS account ID in the line 15 and AWS cross account name in the line 16.

```
1  @Library("TerraformScripts") _
2
3  properties([
4    parameters([
5      choice(name: 'ACCOUNT', choices: ['Dev','Prod'], description: 'Select the Account in which the terraform template needs to be deployed'),
6      choice(name: 'ACTION', choices: ['Create-stack','Delete-stack'], description: 'Select the choice if you want create or delete the terraform stack'
7      string(defaultValue: 'Folder_Name', name: 'FOLDER_NAME', trim: true, description: 'Enter the name of the terraform resource folder name which is i
8      string(defaultValue: 'Folder_Name', name: 'VARIABLE_FOLDER_NAME', trim: true, description: 'Enter the name of the terraform variables folder name
9    ])
10 ])
11
12 pipeline{
13   agent any
14   environment {
15     PROD_ACC_NO=''
16     PROD_IAM_ROLE_NAME=''
17   }
```

Step6: [Optional] If you are maintaining different environments in same AWS account then you can comment from line 14 to line 17 and remove line 23 and adjust the brackets accordingly.

Step7: Now upload the “vars” and “Jenkinsfile.groovy” to the repo which is in the zip file.



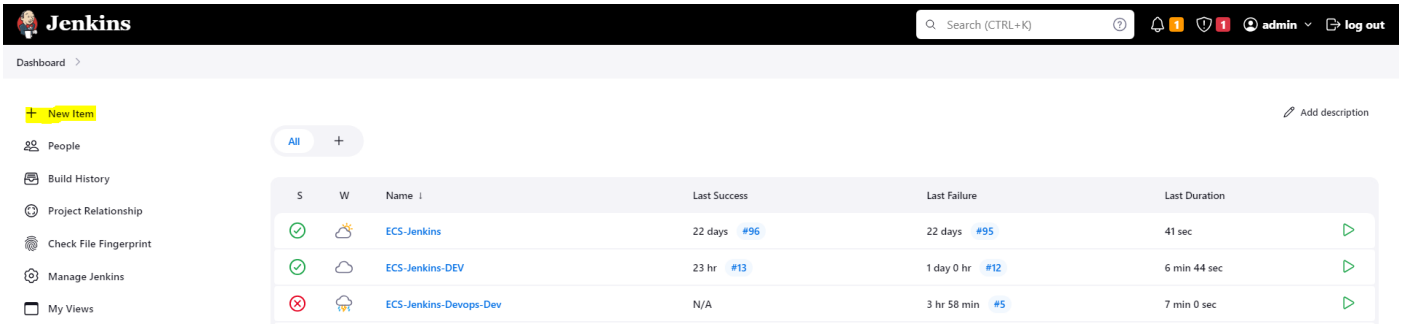
JENKINS JOB CONFIGURATION

DESCRIPTION

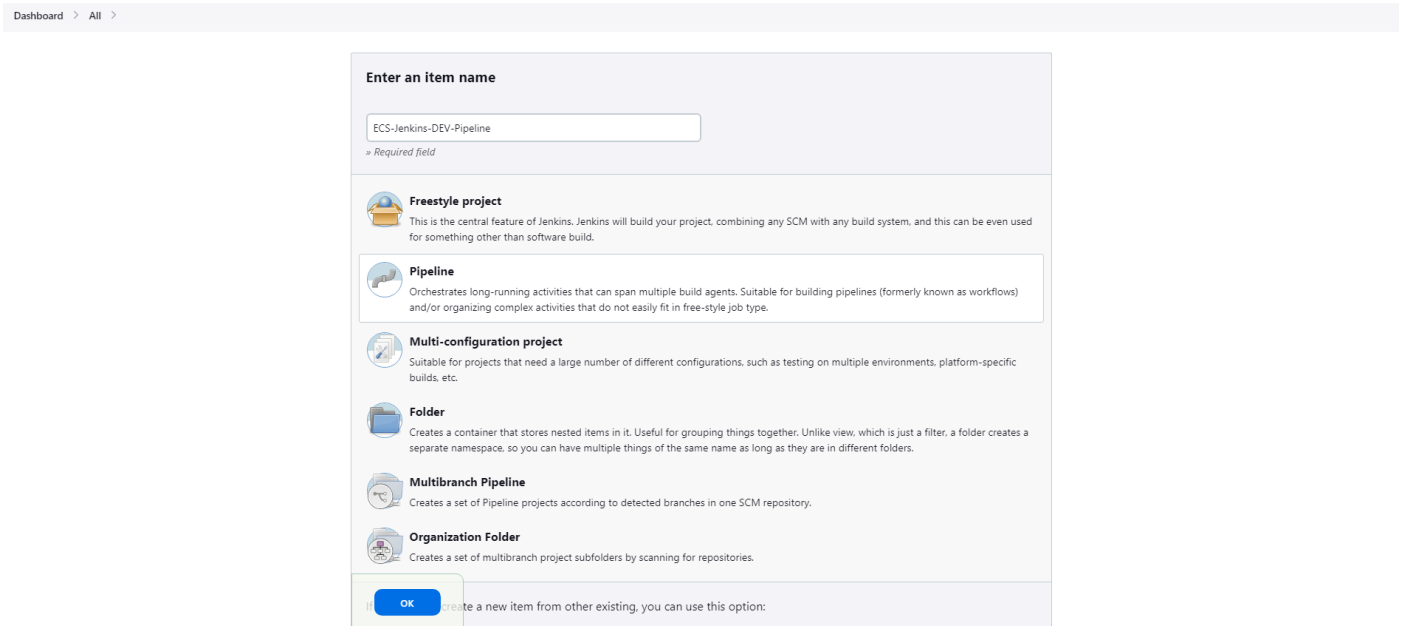
This section guides you on how to configure the Jenkins Job.

PROCEDURE

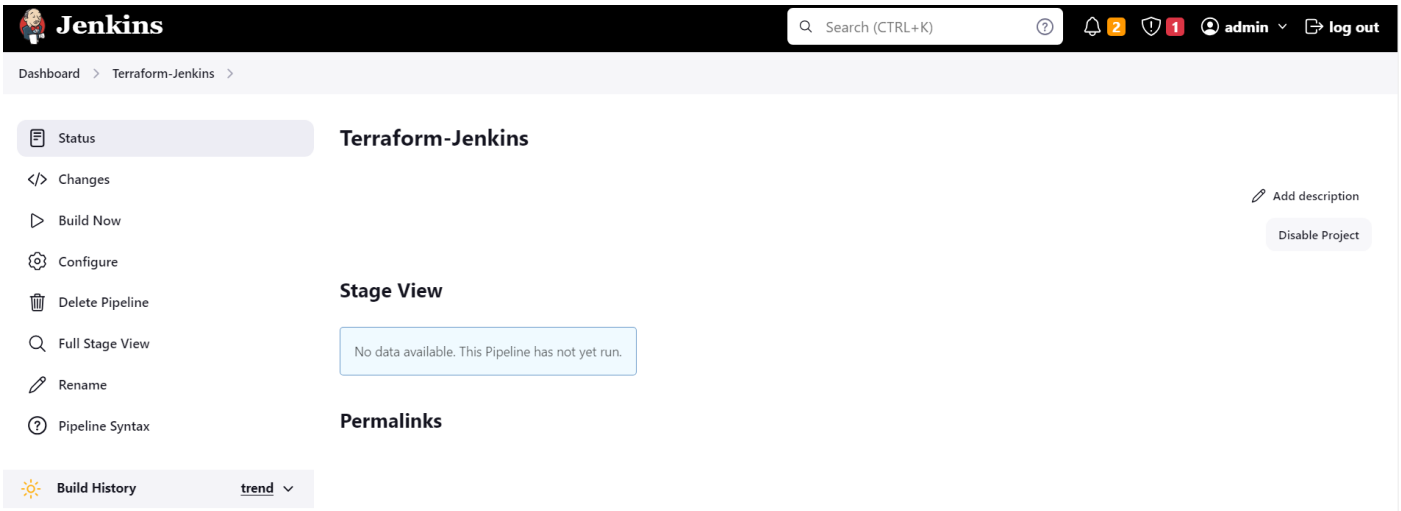
Step1: Login to Jenkins console and click on “New Item”



Step2: Give a Name of your choice and select Pipeline and then click on “OK”.

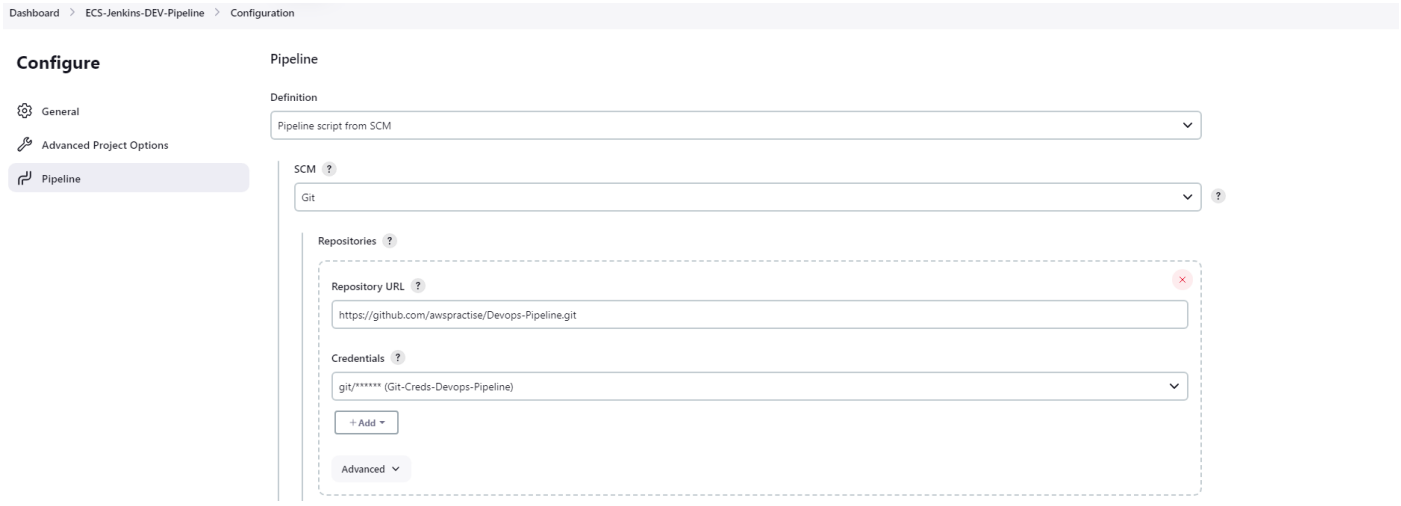


Step3: Click on Configure.

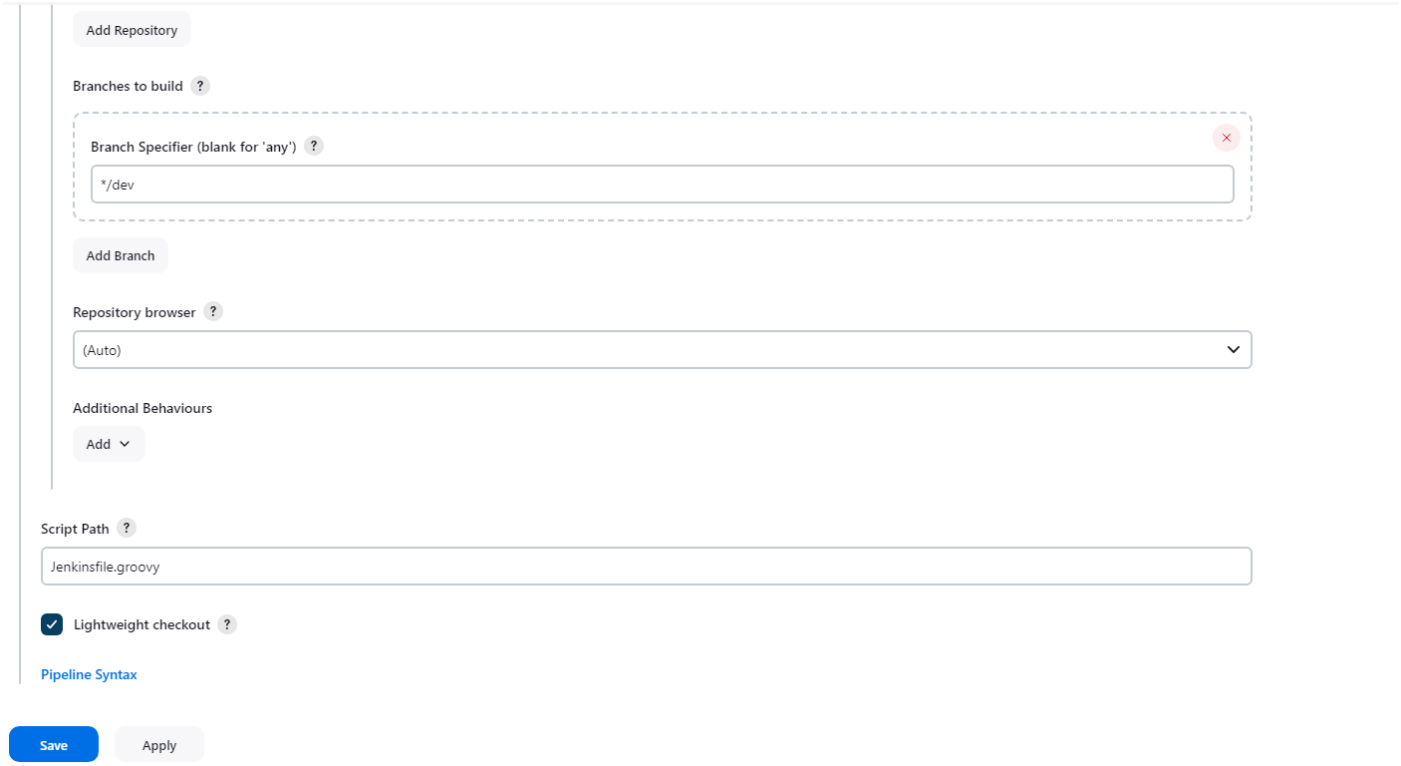


Step4: Under the Pipeline Section Select Definition as “Pipeline script from SCM” and under SCM select “Git”. Next under Repositories give the Terraform Pipeline Repository URL where the

Jenkinsfile.groovy has been uploaded and select the Credentials for it (Refer to Step6 from the Creation of the Jenkins Secret).



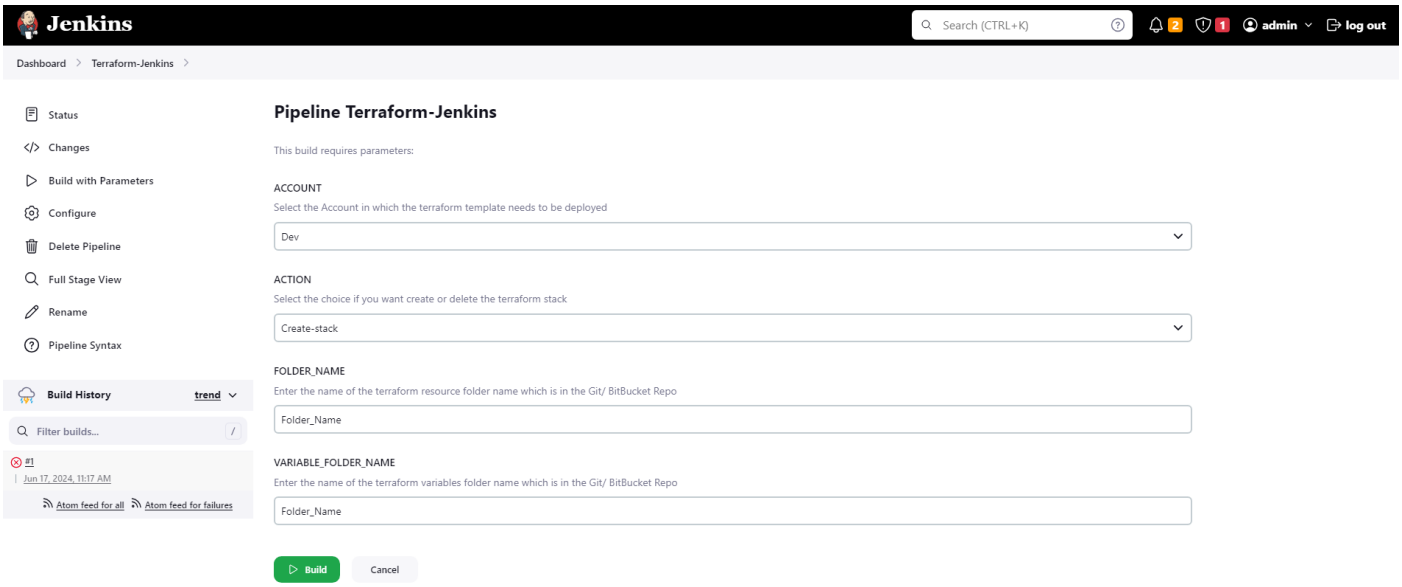
Step5: From Branches to Build specify your branch name of the Terraform Jenkins Pipeline Repository. And for the Script Path give it as “Jenkinsfile.groovy”.



Step6: Once it is done click on “Save” to save the Job.

Step7: Now run the pipeline. The first time you run the pipeline the Job will fail as the Pipeline will not be know the parameters which needs to be passed.

Step8: Once the first job is failed. Again, run the pipeline now the you will be displayed with a menu of Parameters.



Step9: In the above parameters.

I. ACCOUNT: Select in which account you want to deploy the AWS Resource.
II. ACTION: Select Create-stack or Delete-stack
III. FOLDER_NAME: Here give the folder path which has the terraform main.tf file
Ex: “modules/EKS”
VARIABLE_FOLDER_NAME”: Here give the folder path which has the terraform terraform.tfvars and state-lock.tf file
Ex: “terraform-variables/DEV/EKS”

Step10: Once the values have been given properly now run the Jenkins Job to deploy your AWS Resource.